

Control Structures in Python

Technical Notes

Rodrigo Kang

Table of contents

1	Introduction	3
2	Conditions and Boolean Context	4
2.1	Truth Values	5
3	Code Blocks and Indentation	7
4	Conditional Statements	8
4.1	The if Statement	8
4.2	The if-else Statement	9
4.3	The if-elif-else Statement	11
4.4	Independent Conditions versus Mutually Exclusive Branches	13
5	Nested Conditional Statements	14
6	Conditional Expressions	15
7	The pass Statement	17
8	Iteration	18
9	The for Loop	18
9.1	Iterables and Iteration Variables	20
10	The range() Function	22
10.1	The step Argument	24
10.2	Iterating Directly versus Iterating over Indices	25
11	The while Loop	26
11.1	State and Termination	27
12	Choosing between for and while	28
13	The break Statement	30

14 The continue Statement	31
15 The else Clause in Loops	33
16 Nested Loops	35
16.1 break and continue in Nested Loops	37
17 Common Control-Flow Considerations	38
17.1 Off-by-One Errors	38
17.2 Infinite while Loops	39
17.3 Modifying a Collection during Iteration	40
17.4 Avoiding Unnecessary Loops	40
18 Common Iteration Patterns	41
19 Iterating with enumerate()	41
19.1 Changing the Starting Index	43
20 Parallel Iteration with zip()	43
20.1 Iterables of Different Lengths	45
21 Unpacking during Iteration	46
22 Control-Flow Patterns in Data Processing	47
22.1 Validation before Processing	47
22.2 Searching for a Satisfying Element	48
22.3 Iteration until Convergence	48
23 Readability and Control Flow	50
23.1 Prefer Direct Conditions	50
23.2 Avoid Unnecessary Nesting	51
23.3 Do Not Use break or continue without a Clear Purpose	52
23.4 Express Intent rather than Mechanics	53
24 Problems	53
24.1 Problem 1 — Truth Values	53
24.2 Problem 2 — Independent Conditions	54
24.3 Problem 3 — Ordering Conditions	54
24.4 Problem 4 — Conditional Expression	55
24.5 Problem 5 — Point inside a Rectangle	55
24.6 Problem 6 — Leap Year	55
24.7 Problem 7 — Predicting a for Loop	56
24.8 Problem 8 — range()	56
24.9 Problem 9 — Off-by-One Error	56
24.10 Problem 10 — Accumulation	57
24.11 Problem 11 — Arithmetic Progression	57
24.12 Problem 12 — Counting Characters	57
24.13 Problem 13 — Reverse a String	57

24.14	Problem 14 — Filtering with continue	58
24.15	Problem 15 — First Matching Value	58
24.16	Problem 16 — for-else	58
24.17	Problem 17 — Infinite Loop Diagnosis	58
24.18	Problem 18 — Iteration until Convergence	59
24.19	Problem 19 — break and State	59
24.20	Problem 20 — continue	60
24.21	Problem 21 — Nested Loops	60
24.22	Problem 22 — break in Nested Loops	60
24.23	Problem 23 — Matrix Traversal	61
24.24	Problem 24 — Pairwise Comparisons	61
24.25	Problem 25 — enumerate()	61
24.26	Problem 26 — zip()	62
24.27	Problem 27 — Different Lengths with zip()	62
24.28	Problem 28 — Data Validation	62
24.29	Problem 29 — Threshold Classification	63
24.30	Problem 30 — Early Stopping	63
24.31	Problem 31 — Search without a Flag	64
24.32	Problem 32 — Reasoning about Control Flow	64
25	Summary	65

1 Introduction

A program is generally not executed as an unconditional sequence of instructions. Its behaviour often depends on the values encountered during execution: a model may be fitted only if the training data satisfy certain requirements, an iterative algorithm may continue until a convergence criterion is reached, or an observation may be processed differently depending on its characteristics.

Control structures determine which statements are executed and how many times they are executed. They therefore control the **flow of execution** of a program.

Python provides two fundamental forms of control flow:

- **conditional execution**, which determines whether a block of code should be executed according to one or more conditions;
- **iteration**, which repeatedly executes a block of code, either over the elements of an iterable or while a condition remains satisfied.

These mechanisms are implemented primarily through statements such as `if`, `elif`, `else`, `for`, and `while`. Additional statements such as `break`, `continue`, and `pass` provide more specific control over execution.

Before studying these structures, it is useful to understand how Python interprets conditions and how blocks of code are defined.

2 Conditions and Boolean Context

Control-flow statements frequently need to decide whether a condition is satisfied. Conceptually, such a condition is evaluated in a **Boolean context**, in which Python determines whether an expression should be regarded as true or false.

The simplest case is an expression whose result is already a Boolean value:

```
1 x = 7
2
3 x > 5
```

True

The comparison `x > 5` evaluates to True, so it can be used directly as a condition:

```
1 x = 7
2
3 if x > 5:
4     print("x is greater than 5")
```

x is greater than 5

Conditions can be constructed using comparison operators such as

Operator	Meaning
<code>==</code>	equal to
<code>!=</code>	not equal to
<code><</code>	less than
<code><=</code>	less than or equal to
<code>></code>	greater than
<code>>=</code>	greater than or equal to

Table 1: Common comparison operators in Python.

and combined using the Boolean operators and, or, and not.

For example:

```
1 x = 7
2
3 x >= 0 and x <= 10
```

True

Python supports **chained comparisons**, so the previous condition can be expressed more naturally as

```
1 0 <= x <= 10
```

True

This notation corresponds closely to the mathematical expression

$$0 \leq x \leq 10.$$

It is generally preferable to writing

```
1 x >= 0 and x <= 10
```

when both expressions represent the same logical condition.

2.1 Truth Values

A condition does not necessarily have to produce the Boolean objects True or False explicitly. Python can evaluate objects of many types in a Boolean context.

For example:

```
1 bool(0), bool(1)
```

(False, True)

```
1 bool("", bool("Python"))
```

(False, True)

```
1 bool([], bool([1, 2, 3]))
```

(False, True)

As a general rule, objects such as the following are considered false:

- False;
- None;
- numeric zero values, such as 0 and 0.0;
- empty sequences and collections, such as "", [], (), {}, and set().

Most other objects are considered true.

This allows conditions to be written directly in terms of the object being tested. For example, rather than writing

```
1 values = [2, 4, 6]
2
3 if len(values) > 0:
4     print("The collection is not empty")
```

we can write

```
1 values = [2, 4, 6]
2
3 if values:
4     print("The collection is not empty")
```

The collection is not empty

The second form expresses the intended condition directly: execute the block if values is non-empty.

This distinction between **truth values** and explicit comparisons with True or False is important in Python. In most cases, a Boolean expression should be used directly:

```
1 if condition:
2     ...
```

rather than

```
1 if condition == True:
2     ...
```

The first form is simpler and reflects the Boolean semantics of the expression itself.

3 Code Blocks and Indentation

Control structures operate on **blocks of statements**. Unlike languages that delimit blocks using braces or explicit keywords, Python uses indentation as part of its syntax.

Consider:

```
1 x = 8
2
3 if x > 0:
4     print("x is positive")
5     print("This statement belongs to the if block")
6
7 print("This statement is outside the if block")
```

```
x is positive
This statement belongs to the if block
This statement is outside the if block
```

The two indented statements belong to the same block and are therefore controlled by the if statement. The final `print()` statement is not indented and is executed independently of the condition.

The colon `:` marks the beginning of the suite associated with a compound statement, while indentation determines which statements belong to that suite.

The conventional indentation level in Python is **four spaces**. More importantly, indentation must be structurally consistent. It is not merely a formatting convention: changing the indentation can change the meaning of a program or make it syntactically invalid.

This becomes particularly important when control structures are nested:

```

1 x = 8
2
3 if x > 0:
4     print("x is positive")
5
6     if x % 2 == 0:
7         print("x is also even")

```

```

x is positive
x is also even

```

The second if belongs to the block controlled by the first if; consequently, it is evaluated only when $x > 0$ is true.

4 Conditional Statements

Conditional statements allow different parts of a program to be executed according to the result of one or more conditions.

The basic conditional structure in Python is the if statement, which can be extended using elif and else.

4.1 The if Statement

The simplest form of conditional execution is

```

1 if condition:
2     statement

```

The condition is evaluated in a Boolean context. If it is true, the indented block is executed. If it is false, the block is skipped and execution continues with the first statement following the conditional structure.

For example, suppose that a numerical algorithm produces an error measure and we want to report when that error is below a specified tolerance:


```

1 error = 0.003
2 tolerance = 0.01
3
4 if error < tolerance:
5     print("Tolerance satisfied")

```

Tolerance satisfied

The logical flow can be represented as

$$\text{condition} \longrightarrow \begin{cases} \text{execute block,} & \text{if true,} \\ \text{skip block,} & \text{if false.} \end{cases}$$

The if statement does not require an alternative branch. It simply makes the execution of a block conditional.

Multiple statements may belong to the same branch:

```

1 error = 0.003
2 tolerance = 0.01
3
4 if error < tolerance:
5     converged = True
6     print("Tolerance satisfied")
7     print("The algorithm has converged")

```

Tolerance satisfied

The algorithm has converged

All statements at the same indentation level belong to the if block.

4.2 The if-else Statement

Frequently, a program must choose between two mutually exclusive alternatives. The else clause defines the block that is executed when the condition of the corresponding if statement is false.

Its general structure is

```

1 if condition:
2     statements_if_true
3 else:
4     statements_if_false

```

For example:

```

1 error = 0.03
2 tolerance = 0.01
3
4 if error < tolerance:
5     print("The algorithm has converged")
6 else:
7     print("Further iterations are required")

```

Further iterations are required

Exactly one of the two branches is executed.

Conceptually,

$$\text{condition} \longrightarrow \begin{cases} \text{first block,} & \text{if true,} \\ \text{second block,} & \text{if false.} \end{cases}$$

The branches should represent complementary outcomes of the same decision. This often makes an if-else structure clearer than two independent if statements.

For example:

```

1 if score >= threshold:
2     prediction = 1
3
4 if score < threshold:
5     prediction = 0

```

can be written more clearly as

```

1 if score >= threshold:
2     prediction = 1
3 else:
4     prediction = 0

```

The second version makes explicit that the two alternatives belong to a single decision.

4.3 The if-elif-else Statement

A decision may involve more than two possible outcomes. Python provides the `elif` clause—short for *else if*—to construct a sequence of mutually exclusive conditions.

The general form is

```
1 if condition_1:
2     statements_1
3 elif condition_2:
4     statements_2
5 elif condition_3:
6     statements_3
7 else:
8     default_statements
```

Python evaluates the conditions **from top to bottom**. As soon as one condition evaluates to true, its corresponding block is executed and the remainder of the conditional chain is skipped.

Consider a simple classification of a model error:

```
1 error = 0.08
2
3 if error < 0:
4     print("Invalid error value")
5 elif error < 0.05:
6     print("Low error")
7 elif error < 0.10:
8     print("Moderate error")
9 else:
10    print("High error")
```

Moderate error

Since error is 0.08:

1. `error < 0` is false;
2. `error < 0.05` is false;
3. `error < 0.10` is true.

The third block is therefore executed, and the final else is not considered.

This reveals an important property of an if-elif-else chain: **the order of the conditions matters**.

For example, reversing two of the previous conditions would change the logic:

```
1 error = 0.03
2
3 if error < 0.10:
4     print("Moderate error")
5 elif error < 0.05:
6     print("Low error")
```

Moderate error

Although $\text{error} < 0.05$ is also true, that condition is never evaluated because the first condition has already succeeded.

Therefore, when conditions overlap, they should be ordered deliberately. In threshold-based classifications, this often means progressing from the most restrictive condition to increasingly general ones.

The final else clause is optional. When it is present, it acts as a fallback for every case not captured by the preceding conditions.

For example:

```
1 temperature = 21
2
3 if temperature < 10:
4     category = "low"
5 elif temperature < 25:
6     category = "moderate"
7 else:
8     category = "high"
9
10 category
```

'moderate'

The conditions effectively partition the possible values of temperature into three mutually exclusive regions:

$$(-\infty, 10), \quad [10, 25), \quad [25, \infty).$$

4.4 Independent Conditions versus Mutually Exclusive Branches

Several consecutive if statements are not equivalent to an if-elif chain.

Consider:

```
1 x = 12
2
3 if x > 0:
4     print("Positive")
5
6 if x % 2 == 0:
7     print("Even")
```

Positive
Even

Both conditions are evaluated independently, so both messages are printed.

By contrast:

```
1 x = 12
2
3 if x > 0:
4     print("Positive")
5 elif x % 2 == 0:
6     print("Even")
```

Positive

only prints "Positive". Once the first condition is satisfied, Python skips the remaining branches.

The appropriate structure therefore depends on the intended logic:

- use **independent if statements** when several conditions may legitimately hold and several blocks may need to execute;

- use an **if-elif-else chain** when the branches represent alternatives and only one should be selected.

This distinction is particularly important when translating a mathematical or business rule into code. The syntax may be similar, but the logical models are different.

5 Nested Conditional Statements

Conditional statements can be placed inside other conditional statements. This creates **nested conditionals**.

For example:

```
1 value = 12
2
3 if value >= 0:
4     if value == 0:
5         print("Zero")
6     else:
7         print("Positive")
8 else:
9     print("Negative")
```

Positive

The inner conditional is evaluated only when the outer condition `value >= 0` is satisfied.

Nested structures are useful when a second decision is meaningful only after a first condition has been established. For instance:

```
1 model_fitted = True
2 validation_score = 0.87
3
4 if model_fitted:
5     if validation_score >= 0.80:
6         print("Model ready for further evaluation")
7     else:
8         print("Model requires revision")
9 else:
10    print("Fit the model first")
```

Model ready for further evaluation

However, excessive nesting can make the flow of execution difficult to follow. When conditions can be expressed naturally as a single Boolean expression, combining them may produce clearer code.

For example,

```
1 if x > 0:
2     if x < 10:
3         print("x belongs to the interval")
```

can be written as

```
1 x = 7
2
3 if 0 < x < 10:
4     print("x belongs to the interval")
```

x belongs to the interval

These forms are logically equivalent for this case, but the second expresses the condition more directly.

Nesting should therefore represent a genuine hierarchy of decisions rather than being used simply because multiple conditions are involved.

6 Conditional Expressions

Python also provides a compact expression for selecting between two values according to a condition:

```
1 value_if_true if condition else value_if_false
```

This is called a **conditional expression**. It is also informally known as the *ternary operator* because it involves three components: the expression returned when the condition is true, the condition itself, and the expression returned otherwise.

For example:

```

1 x = 7
2
3 sign = "non-negative" if x >= 0 else "negative"
4
5 sign

```

'non-negative'

The expression

```

1 "non-negative" if x >= 0 else "negative"

```

evaluates to one of two values and that result is assigned to sign.

An equivalent if-else statement would be

```

1 x = 7
2
3 if x >= 0:
4     sign = "non-negative"
5 else:
6     sign = "negative"
7
8 sign

```

'non-negative'

The distinction is worth emphasizing. An if statement controls whether **statements or blocks of statements** are executed, whereas a conditional expression **produces a value**.

Conditional expressions are particularly useful for short assignments:

```

1 score = 0.83
2 threshold = 0.80
3
4 label = "accepted" if score >= threshold else "rejected"
5
6 label

```

'accepted'

They should generally remain simple. Although conditional expressions can themselves be nested, code such as

```
1 category = "low" if x < 0 else "medium" if x < 10 else "high"
```

is often less readable than the equivalent if-elif-else statement:

```
1 if x < 0:
2     category = "low"
3 elif x < 10:
4     category = "medium"
5 else:
6     category = "high"
```

Compact syntax is useful only when it makes the intended logic at least as clear as the expanded form.

7 The pass Statement

Python requires a syntactically valid statement inside a block. Occasionally, however, we want to define the structure of a program without performing any action yet.

The pass statement performs no operation:

```
1 x = 5
2
3 if x > 0:
4     pass
5 else:
6     print("x is not positive")
```

The pass statement is therefore a **null statement**: Python executes it and immediately continues with the next statement.

It is commonly useful as a temporary placeholder while developing code:

```
1 if condition:
2     # Implementation to be added later
3     pass
```

It can also be used in other contexts that require a non-empty block, such as loops, functions, and classes.

`pass` should not be confused with `continue`, which will be introduced later. `pass` does not alter the normal flow of execution; `continue` explicitly terminates the current iteration of a loop and proceeds to the next one.

8 Iteration

Conditional statements determine **whether** a block of code is executed. A second fundamental form of control flow determines **how many times** a block is executed.

Iteration is the repeated execution of a block of statements. In Python, iteration is primarily implemented through two types of loops:

- a for loop iterates over the elements produced by an iterable;
- a while loop repeatedly executes a block while a condition remains true.

Although both structures can sometimes be used to solve the same problem, they express different ideas. A for loop is generally appropriate when execution is organised around the elements of an iterable, whereas a while loop is appropriate when repetition is controlled primarily by a condition.

9 The for Loop

The for statement is Python's fundamental construct for iterating over the elements of an iterable.

Its general form is

```
1 for variable in iterable:
2     statements
```

At each iteration, Python obtains the next element from the iterable, assigns it to variable, and executes the indented block. Iteration continues until no elements remain.

For example:

```
1 values = [2, 4, 6, 8]
2
3 for value in values:
4     print(value)
```

2
4
6
8

The variable value successively refers to

2, 4, 6, 8.

The same mechanism applies to other iterable objects. Strings, for example, can be iterated character by character:

```
1 word = "Python"
2
3 for character in word:
4     print(character)
```

P
y
t
h
o
n

The important idea is not that a for loop increments a counter automatically. Rather, Python repeatedly retrieves values from an **iterable**.

This distinguishes Python's for statement from the counter-based loops commonly found in some other programming languages.

For example, suppose that we want to compute the sum of a collection of values:

```
1 values = [3, 7, 2, 5]
2
3 total = 0
4
5 for value in values:
6     total += value
7
8 total
```

17

The execution can be viewed as a sequence of updates:

$$\begin{aligned}S_0 &= 0, \\S_1 &= S_0 + 3, \\S_2 &= S_1 + 7, \\S_3 &= S_2 + 2, \\S_4 &= S_3 + 5.\end{aligned}$$

At the end of the loop,

$$S_4 = 17.$$

Here, `total` is an **accumulator**: its value is updated at every iteration using information from the current element.

This pattern occurs frequently in computational problems, although built-in functions or specialised numerical operations are often preferable when they already provide the required operation. For example, the previous calculation could simply be written as

```
1 sum(values)
```

17

Understanding the explicit loop remains important because many algorithms require more complex operations than a single built-in function can express.

9.1 Iterables and Iteration Variables

The object appearing after `in` must provide values that Python can iterate over. Such an object is called an **iterable**.

Common iterable objects include

- strings;
- lists;
- tuples;
- dictionaries;
- sets;
- range objects;
- many objects provided by scientific and data-processing libraries.

Consider:

```
1 models = ["linear regression", "decision tree", "random forest"]
2
3 for model in models:
4     print(model)
```

linear regression
decision tree
random forest

The iteration variable `model` is assigned a different element of `models` during each iteration.

The name of the iteration variable has no special significance to Python:

```
1 for x in models:
2     print(x)
```

linear regression
decision tree
random forest

works in exactly the same way. However, descriptive names generally make the meaning of the loop clearer.

The iteration variable continues to exist after the loop and normally retains the value assigned during the final iteration:

```
1 values = [10, 20, 30]
2
3 for value in values:
4     pass
5
6 value
```

30

Although this behaviour is sometimes useful to know, code should rarely depend on the final value of an iteration variable unless that intention is explicit.

10 The range() Function

A common requirement is to iterate over a sequence of integers. Python provides the `range()` function for this purpose.

Its most general form is

```
1 range(start, stop, step)
```

where

- start specifies the first value;
- stop defines the boundary at which the sequence stops;
- step specifies the increment between consecutive values.

The value specified by stop is **not included**.

For example:

```
1 list(range(1, 6))
```

```
[1, 2, 3, 4, 5]
```

produces the integers

1, 2, 3, 4, 5.

Thus,

```
1 for i in range(1, 6):  
2     print(i)
```

```
1  
2  
3  
4  
5
```

executes five iterations.

If only one argument is supplied,

```
1 range(stop)
```

the starting value defaults to 0 and the step defaults to 1.

For example:

```
1 list(range(5))
```

[0, 1, 2, 3, 4]

produces

0, 1, 2, 3, 4.

Consequently,

```
1 for i in range(5):  
2     print(i)
```

0
1
2
3
4

performs five iterations even though the final value of *i* is 4.

This half-open convention,

[start, stop),

is widespread in Python. It is particularly convenient when working with zero-based indices because

```
1 range(n)
```

produces exactly

$0, 1, \dots, n - 1,$

which are the valid positional indices of a sequence of length *n*.

10.1 The step Argument

The optional step argument determines the difference between consecutive values.

For example:

```
1 list(range(0, 11, 2))
```

[0, 2, 4, 6, 8, 10]

produces

0, 2, 4, 6, 8, 10.

The step may also be negative:

```
1 list(range(5, 0, -1))
```

[5, 4, 3, 2, 1]

which produces

5, 4, 3, 2, 1.

A negative step requires a starting value greater than the stopping boundary. For example:

```
1 for i in range(5, 0, -1):  
2     print(i)
```

5
4
3
2
1

can be used for a countdown.

`range()` does not need to construct a list containing all the integers before iteration begins. It represents the integer sequence compactly and produces the required values during iteration.

10.2 Iterating Directly versus Iterating over Indices

Suppose we have

```
1 values = [4.2, 7.1, 3.8]
```

One possibility is to iterate over positions:

```
1 values = [4.2, 7.1, 3.8]
2
3 for i in range(len(values)):
4     print(values[i])
```

```
4.2
7.1
3.8
```

However, when the index itself is not needed, Python allows the more direct form

```
1 for value in values:
2     print(value)
```

```
4.2
7.1
3.8
```

The second version better expresses the intended operation: process each value in the collection.

Using indices is appropriate when the position itself is required, when different sequences must be accessed according to corresponding positions, or when the algorithm genuinely operates on indices. It should not be introduced merely to reproduce the behaviour of iteration available directly through the iterable.

Later we will see that Python provides constructs such as `enumerate()` and `zip()` for many cases in which indices might otherwise be used explicitly.

11 The while Loop

A while loop repeatedly executes a block of statements **while a condition remains true**.

Its general form is

```
1 while condition:
2     statements
```

Before every iteration, Python evaluates condition in a Boolean context:

- if the condition is true, the loop body is executed;
- after the body finishes, the condition is evaluated again;
- when the condition becomes false, execution continues after the loop.

For example:

```
1 i = 1
2
3 while i <= 5:
4     print(i)
5     i += 1
```

1
2
3
4
5

The state of i evolves as

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6.$$

When i becomes 6, the condition

```
1 i <= 5
```

is false and the loop terminates.

Unlike a for loop, a while loop does not inherently advance through an iterable. Its continuation depends entirely on the Boolean condition.

11.1 State and Termination

A while loop normally involves some **state** that changes during execution.

Consider again:

```
1 i = 1
2
3 while i <= 5:
4     print(i)
5     i += 1
```

The statement

```
1 i += 1
```

changes the state on which the condition depends. Without this update,

```
1 i = 1
2
3 while i <= 5:
4     print(i)
```

the condition would remain true indefinitely.

This produces an **infinite loop**.

Not every terminating while loop must modify a simple counter, but something affecting the continuation condition must eventually change if the loop is expected to terminate.

This is particularly important in iterative numerical algorithms. Consider a simplified convergence process:

```
1 error = 1.0
2 tolerance = 0.01
3
4 while error > tolerance:
5     error *= 0.5
6     print(error)
```

0.5
0.25
0.125
0.0625
0.03125
0.015625
0.0078125

At each iteration,

$$e_{k+1} = \frac{1}{2}e_k,$$

so that

$$e_k = \left(\frac{1}{2}\right)^k e_0.$$

As the error decreases, the condition eventually becomes false and the loop terminates.

This illustrates a natural use of while: the number of iterations need not be specified in advance. Execution continues until a state-dependent criterion is satisfied.

12 Choosing between for and while

A for loop and a while loop can sometimes express the same computation.

For example:

```
1 for i in range(1, 6):  
2     print(i)
```

1
2
3
4
5

and

```
1 i = 1
2
3 while i <= 5:
4     print(i)
5     i += 1
```

```
1
2
3
4
5
```

produce the same sequence of values.

However, the two versions express different control-flow models.

The for version says:

iterate over the integers from 1 through 5.

The while version says:

continue executing while $i \leq 5$.

When the computation is fundamentally based on traversing an iterable or executing a known collection of iterations, for is generally clearer.

When termination depends on a dynamically changing condition rather than on exhausting an iterable, while is generally more natural.

A useful conceptual distinction is therefore:

```
1 for -> iteration over values
2 while -> repetition while a condition holds
```

This is not an absolute rule, but it is a useful default when choosing between the two structures.

13 The break Statement

Normally, a loop terminates when

- a for loop exhausts its iterable, or
- the condition of a while loop becomes false.

The break statement allows the current loop to be terminated immediately.

For example:

```
1 values = [0.4, 0.7, 1.2, -0.3, 0.8]
2
3 for value in values:
4     if value < 0:
5         print("Negative value found")
6         break
7
8     print(value)
```

```
0.4
0.7
1.2
Negative value found
```

Once the negative value is encountered, break terminates the for loop. The remaining elements are not processed.

This pattern is useful when continuing the search or computation would no longer provide useful information.

For example, suppose we want to determine the first iteration at which an error falls below a tolerance:

```
1 errors = [0.42, 0.19, 0.08, 0.03, 0.009, 0.004]
2 tolerance = 0.01
3
4 for iteration in range(len(errors)):
5     if errors[iteration] < tolerance:
6         print(f"Convergence reached at iteration {iteration}")
7         break
```

```
Convergence reached at iteration 4
```

Once convergence is detected, there is no need for the loop to continue.

The same principle applies to while loops:

```
1 iteration = 0
2 error = 1.0
3
4 while True:
5     error *= 0.5
6     iteration += 1
7
8     if error < 0.01:
9         break
10
11 print(iteration, error)
```

7 0.0078125

Here, while True creates a loop whose condition is deliberately always true. Termination is instead controlled explicitly by break.

This pattern can be appropriate when the termination condition is most naturally evaluated inside the body of the loop. However, it should be used deliberately: if no reachable break is executed, the loop is infinite.

14 The continue Statement

The continue statement also changes the normal flow of a loop, but unlike break, it does not terminate the loop.

Instead, continue terminates the **current iteration** and proceeds to the next one.

For example:

```
1 values = [3.2, -1.0, 4.5, -0.2, 2.1]
2
3 for value in values:
4     if value < 0:
5         continue
6
7     print(value)
```

3.2
4.5
2.1

When value is negative, execution immediately moves to the next element. The `print()` statement is therefore executed only for non-negative values.

This can be useful when certain observations should be skipped:

```
1 observations = [1.8, None, 2.4, None, 3.1]
2
3 for observation in observations:
4     if observation is None:
5         continue
6
7     print(observation ** 2)
```

3.24
5.76
9.610000000000001

Conceptually, `break` and `continue` have different effects:

Statement	Effect
<code>break</code>	terminates the loop
<code>continue</code>	terminates only the current iteration

Table 2: Effect of `break` and `continue` inside a loop.

Code using `continue` can often be written without it. For example,

```
1 for value in values:
2     if value <= 0:
3         print(value)
```

may be preferable to


```
1 for value in values:
2     if value < 0:
3         continue
4
5     print(value)
```

when the first form expresses the logic more directly.

`continue` becomes particularly useful when skipping an invalid or irrelevant case early prevents the remainder of the loop body from becoming deeply nested.

15 The else Clause in Loops

Python allows both `for` and `while` loops to have an `else` clause.

For example:

```
1 for i in range(3):
2     print(i)
3 else:
4     print("Loop completed")
```

```
0
1
2
Loop completed
```

and

```
1 i = 0
2
3 while i < 3:
4     print(i)
5     i += 1
6 else:
7     print("Loop completed")
```

```
0
1
2
Loop completed
```

In both cases, the else block is executed after the loop finishes normally.

The important detail is that **loop else is not simply a block that runs after every loop**. Its behaviour is tied to break.

The else block is executed when the loop terminates **without encountering a break**.

Consider:

```
1 values = [2, 4, 6, 8]
2 target = 5
3
4 for value in values:
5     if value == target:
6         print("Target found")
7         break
8 else:
9     print("Target not found")
```

Target not found

Since 5 is not present, the loop exhausts the iterable normally and the else block executes.

Now consider:

```
1 values = [2, 4, 6, 8]
2 target = 6
3
4 for value in values:
5     if value == target:
6         print("Target found")
7         break
8 else:
9     print("Target not found")
```

Target found

This time, break is executed when 6 is encountered. The else block is therefore skipped.

This structure is particularly useful for search problems:

```

1 for element in iterable:
2     if desired_condition(element):
3         # Found
4         break
5 else:
6     # No element satisfied the condition

```

The interpretation is better expressed as

execute else if the loop completed without being terminated by break

rather than simply

execute else when the loop finishes.

The same semantics apply to while.

For example:

```

1 attempt = 1
2
3 while attempt <= 3:
4     success = attempt == 2
5
6     if success:
7         print("Successful attempt")
8         break
9
10    attempt += 1
11 else:
12    print("No successful attempt")

```

Successful attempt

Because the loop is terminated by break, the else clause is not executed.

16 Nested Loops

A loop may contain another loop. Such structures are called **nested loops**.

Consider:

```

1 for i in range(3):
2     for j in range(2):
3         print(i, j)

```

```

0 0
0 1
1 0
1 1
2 0
2 1

```

For every value of i , the inner loop completes all of its iterations.

The generated pairs are

$(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1).$

If the outer loop has m iterations and the inner loop has n iterations for every outer iteration, the body of the inner loop executes

$$mn$$

times.

This simple observation becomes important when considering computational cost. For example:

```

1 values = [2, 5, 8]
2
3 for x in values:
4     for y in values:
5         print(x, y)

```

```

2 2
2 5
2 8
5 2
5 5
5 8
8 2
8 5
8 8

```

examines every ordered pair of values and therefore performs

$$n^2$$

inner iterations for a collection of length n .

Nested loops arise naturally in problems involving combinations of observations, matrices, grids, parameter combinations, and pairwise comparisons.

For example, a simple traversal of a matrix represented as nested lists can be written as

```
1 matrix = [  
2     [1, 2, 3],  
3     [4, 5, 6],  
4 ]  
5  
6 for row in matrix:  
7     for value in row:  
8         print(value)
```

```
1  
2  
3  
4  
5  
6
```

The outer loop iterates over rows, while the inner loop iterates over the values within each row.

16.1 break and continue in Nested Loops

When `break` is used inside nested loops, it terminates only the **innermost loop containing it**.

For example:

```
1 for i in range(3):  
2     for j in range(3):  
3         if j == 1:  
4             break  
5  
6     print(i, j)
```

```
0 0  
1 0  
2 0
```

The `break` statement terminates the loop over `j`, but the outer loop over `i` continues.

Similarly, `continue` affects only the loop in which it appears:

```
1 for i in range(3):  
2     for j in range(3):  
3         if j == 1:  
4             continue  
5  
6     print(i, j)
```

```
0 0  
0 2  
1 0  
1 2  
2 0  
2 2
```

Here, the iteration with `j == 1` is skipped, but the inner loop continues with the next value of `j`.

This distinction becomes important as nested control structures become more complex. Excessive nesting combined with multiple `break` and `continue` statements can make execution difficult to reason about. In such cases, reorganising the logic or separating part of the computation into functions often produces clearer code.

17 Common Control-Flow Considerations

Although loops are syntactically simple, several recurring issues are worth keeping in mind.

17.1 Off-by-One Errors

Because `range()` excludes its stopping value, incorrect boundaries can cause a loop to execute one iteration too many or one iteration too few.

For example:

```
1 list(range(1, 5))
```

[1, 2, 3, 4]

contains

1, 2, 3, 4,

not 5.

If the intention is to include 5, the stopping boundary must be 6:

```
1 list(range(1, 6))
```

[1, 2, 3, 4, 5]

Errors of this type are commonly called **off-by-one errors**.

17.2 Infinite while Loops

A while condition should be examined together with the state changes occurring inside the loop.

For example,

```
1 x = 1
2
3 while x < 10:
4     print(x)
```

never changes x and therefore never makes the condition false.

A terminating version could be

```
1 x = 1
2
3 while x < 10:
4     print(x)
5     x += 1
```

or termination could be controlled explicitly by break.

17.3 Modifying a Collection during Iteration

Care is required when structurally modifying a collection while iterating over it.

For example, removing elements from a list while traversing that same list can cause elements to be skipped because the positions of the remaining elements change during iteration.

When the intention is to construct a filtered collection, it is generally clearer to create a new collection rather than mutate the iterable being traversed.

For example:

```
1 values = [3, -1, 5, -2, 8]
2
3 non_negative = []
4
5 for value in values:
6     if value >= 0:
7         non_negative.append(value)
8
9 non_negative
```

[3, 5, 8]

Later, we will see more concise Python constructs for expressing transformations and filtering operations, but the explicit loop makes the underlying control flow clear.

17.4 Avoiding Unnecessary Loops

An explicit loop is not always the clearest or most appropriate solution.

For example,

```
1 values = [1, 2, 3, 4]
2
3 total = 0
4
5 for value in values:
6     total += value
```

is valid, but


```
1 total = sum(values)
```

expresses the operation more directly.

Similarly, numerical and data-processing libraries frequently provide vectorised or specialised operations that are preferable to manually iterating over individual observations.

The purpose of understanding loops is therefore not to use them everywhere. It is to understand the control-flow mechanism well enough to recognise when explicit iteration is necessary and when a higher-level operation expresses the computation more clearly.

18 Common Iteration Patterns

The basic syntax of `for` is sufficient to express any ordinary iteration over an iterable. In practice, however, several recurring patterns occur often enough that Python provides constructs specifically designed for them.

Understanding these constructs helps avoid unnecessary indexing and makes the intended relationship between the values being processed more explicit.

19 Iterating with `enumerate()`

Sometimes both the value and its position are required during iteration.

A direct approach is

```
1 values = [4.2, 7.1, 3.8]
2
3 for i in range(len(values)):
4     print(i, values[i])
```

```
0 4.2
1 7.1
2 3.8
```

This works, but the loop is conceptually about the elements of `values`, not about generating integers.

Python provides `enumerate()` for this situation:

```

1 values = [4.2, 7.1, 3.8]
2
3 for i, value in enumerate(values):
4     print(i, value)

```

```

0 4.2
1 7.1
2 3.8

```

At each iteration, `enumerate()` provides a pair containing

1. the current index;
2. the corresponding element.

The sequence of pairs is conceptually

(0, 4.2), (1, 7.1), (2, 3.8).

The assignment

```

1 for i, value in enumerate(values):

```

uses **unpacking**: the two components produced at each iteration are assigned to `i` and `value`, respectively.

For example, suppose we want to identify the first observation exceeding a threshold:

```

1 scores = [0.41, 0.58, 0.73, 0.86, 0.91]
2 threshold = 0.80
3
4 for i, score in enumerate(scores):
5     if score > threshold:
6         print(f"First score above threshold: index {i}, value {score}")
7         break

```

First score above threshold: index 3, value 0.86

Here, both the position and the value are relevant, so `enumerate()` expresses the operation directly.

19.1 Changing the Starting Index

By default, `enumerate()` starts counting at zero. A different starting value can be specified:

```
1 models = ["linear", "tree", "forest"]
2
3 for position, model in enumerate(models, start=1):
4     print(position, model)
```

```
1 linear
2 tree
3 forest
```

This produces positions

1, 2, 3

without changing the underlying collection.

Using `start=1` can be useful when the counter represents a human-readable position rather than a zero-based sequence index.

20 Parallel Iteration with `zip()`

Another common pattern is processing corresponding elements from two or more iterables.

Suppose that

```
1 y_true = [1, 0, 1, 1]
2 y_pred = [1, 1, 1, 0]
```

represent observed and predicted class labels.

One approach would be to iterate over indices:

```
1 for i in range(len(y_true)):
2     print(y_true[i], y_pred[i])
```

Python provides `zip()` to express the correspondence directly:

```

1 y_true = [1, 0, 1, 1]
2 y_pred = [1, 1, 1, 0]
3
4 for observed, predicted in zip(y_true, y_pred):
5     print(observed, predicted)

```

```

1 1
0 1
1 1
1 0

```

Conceptually, `zip()` forms successive tuples

(1, 1), (0, 1), (1, 1), (1, 0),

and the loop unpacks each tuple into `observed` and `predicted`.

This pattern can be used, for example, to count classification errors explicitly:

```

1 y_true = [1, 0, 1, 1]
2 y_pred = [1, 1, 1, 0]
3
4 errors = 0
5
6 for observed, predicted in zip(y_true, y_pred):
7     if observed != predicted:
8         errors += 1
9
10 errors

```

```

2

```

If n observations are processed and m are incorrectly classified, the empirical error rate is

$$\frac{m}{n}.$$

It could therefore be computed explicitly as

```
1 error_rate = errors / len(y_true)
2
3 error_rate
```

0.5

In real machine-learning code, specialised library functions would normally be used for model evaluation. The explicit loop is useful here because it makes the underlying control flow visible.

20.1 Iterables of Different Lengths

By default, `zip()` stops when the shortest iterable is exhausted.

For example:

```
1 a = [1, 2, 3]
2 b = [10, 20]
3
4 for x, y in zip(a, b):
5     print(x, y)
```

1 10
2 20

produces only two pairs.

This behaviour should be understood rather than assumed away. If corresponding collections are expected to have equal lengths, a mismatch may indicate a problem in the data or in the preceding computation.

Modern Python also allows strict checking:

```
1 zip(a, b, strict=True)
```

With `strict=True`, Python raises an exception if the iterables do not have the same length.

This can be useful when unequal lengths represent an invalid state rather than an acceptable truncation.

21 Unpacking during Iteration

The values produced by an iterable can themselves have structure.

Suppose that each observation is represented by a pair:

```
1 points = [(1, 2), (3, 5), (4, 1)]
2
3 for x, y in points:
4     print(x, y)
```

```
1 2
3 5
4 1
```

Each tuple is unpacked directly into x and y.

This can make the relationship between the data and the loop body clearer than repeatedly indexing each tuple:

```
1 for point in points:
2     x = point[0]
3     y = point[1]
```

The same pattern works naturally with collections representing key-value pairs or other structured records.

For example:

```
1 metrics = [
2     ("accuracy", 0.91),
3     ("precision", 0.87),
4     ("recall", 0.84),
5 ]
6
7 for metric, value in metrics:
8     print(metric, value)
```

```
accuracy 0.91
precision 0.87
recall 0.84
```

The loop variables describe the components of each observation directly.

22 Control-Flow Patterns in Data Processing

The individual control structures introduced in this chapter are simple. Their practical importance comes from combining them to express common computational patterns.

22.1 Validation before Processing

A computation may only be meaningful for valid inputs.

For example:

```
1 values = [2.4, 5.1, -1.0, 3.8]
2
3 for value in values:
4     if value < 0:
5         print("Invalid negative value")
6         continue
7
8     print(value ** 0.5)
```

```
1.5491933384829668
2.258317958127243
Invalid negative value
1.9493588689617927
```

The conditional acts as a **guard**: invalid observations are identified before the main computation is performed.

The same idea can be expressed without continue:

```
1 values = [2.4, 5.1, -1.0, 3.8]
2
3 for value in values:
4     if value >= 0:
5         print(value ** 0.5)
```

```
1.5491933384829668
2.258317958127243
1.9493588689617927
```

Which version is preferable depends on the surrounding code. When the main processing block is long, handling exceptional cases early can reduce nesting and make the normal execution path easier to follow.

22.2 Searching for a Satisfying Element

A common pattern is to inspect a collection until an element satisfying a criterion is found:

```
1 scores = [0.41, 0.56, 0.73, 0.88, 0.91]
2
3 for score in scores:
4     if score >= 0.85:
5         print("Suitable score found:", score)
6         break
7 else:
8     print("No score satisfies the criterion")
```

Suitable score found: 0.88

This combines

- iteration;
- conditional execution;
- early termination with `break`;
- loop `else` to represent unsuccessful completion of the search.

The structure is therefore

```
1 for element in iterable:
2     if condition:
3         # successful search
4         break
5 else:
6     # no element satisfied the condition
```

This is a useful Python pattern because it avoids maintaining a separate Boolean flag solely to determine whether the search succeeded.

22.3 Iteration until Convergence

Many numerical algorithms repeatedly update a state until a stopping criterion is satisfied.

A simplified form is


```
1 while error > tolerance:
2     update_state()
3     recompute_error()
```

In practice, relying exclusively on convergence may be unsafe. If convergence does not occur, the loop could continue indefinitely.

A more robust conceptual structure also imposes a maximum number of iterations:

```
1 error = 1.0
2 tolerance = 1e-4
3 iteration = 0
4 max_iterations = 100
5
6 while error > tolerance and iteration < max_iterations:
7     error *= 0.6
8     iteration += 1
9
10 print(iteration, error)
```

```
19 6.0935974001049565e-05
```

The loop terminates when either

$$e_k \leq \varepsilon$$

or

$$k \geq k_{\max},$$

where ε is the tolerance and $k_{\max}$ is the maximum allowed number of iterations.

This illustrates an important general principle: the stopping condition of an iterative algorithm often combines a **desired convergence criterion** with a **safety limit**.

The same logic can be expressed using break:

```

1 error = 1.0
2 tolerance = 1e-4
3 max_iterations = 100
4
5 for iteration in range(1, max_iterations + 1):
6     error *= 0.6
7
8     if error <= tolerance:
9         break
10
11 print(iteration, error)

```

```
19 6.0935974001049565e-05
```

The choice between these forms depends on which control-flow interpretation best reflects the algorithm.

23 Readability and Control Flow

Correct control flow is not only a question of obtaining the expected output. The structure of the code should also make the logic easy to inspect.

23.1 Prefer Direct Conditions

When a Boolean value already expresses the required condition, use it directly.

Prefer

```

1 if is_valid:
2     ...

```

to

```

1 if is_valid == True:
2     ...

```

and prefer

```
1 if not is_valid:
2     ...
```

to

```
1 if is_valid == False:
2     ...
```

Similarly, when checking whether a collection contains elements,

```
1 if values:
2     ...
```

usually communicates the intention more directly than

```
1 if len(values) > 0:
2     ...
```

23.2 Avoid Unnecessary Nesting

Deeply nested conditionals can obscure the main execution path.

For example:

```
1 if value is not None:
2     if value >= 0:
3         if value <= 1:
4             process(value)
```

may be expressed more directly as

```
1 if value is not None and 0 <= value <= 1:
2     process(value)
```

when the conditions belong naturally to the same logical test.

In longer computations, handling invalid cases early can also reduce nesting:

```
1 for value in values:
2     if value is None:
3         continue
4
5     if value < 0:
6         continue
7
8     process(value)
```

instead of

```
1 for value in values:
2     if value is not None:
3         if value >= 0:
4             process(value)
```

The objective is not to minimise the number of lines, but to make the control flow easy to understand.

23.3 Do Not Use `break` or `continue` without a Clear Purpose

`break` and `continue` are useful when they simplify the logical structure of a loop. Used excessively, they can make the path through a loop difficult to trace.

For a short loop such as

```
1 for value in values:
2     if value >= 0:
3         process(value)
```

introducing `continue`

```
1 for value in values:
2     if value < 0:
3         continue
4
5     process(value)
```

does not necessarily improve the code.

The second form becomes more useful when several statements follow the validation step and avoiding additional indentation makes the main processing path clearer.

23.4 Express Intent rather than Mechanics

Python frequently provides a direct way to express a common operation.

For example:

```
1 for i in range(len(values)):
2     print(i, values[i])
```

works, but

```
1 for i, value in enumerate(values):
2     print(i, value)
```

better expresses the fact that both the position and value are required.

Likewise,

```
1 for i in range(len(x)):
2     process(x[i], y[i])
```

can often be written as

```
1 for x_i, y_i in zip(x, y):
2     process(x_i, y_i)
```

The preferred form is generally the one that communicates the structure of the computation most directly.

24 Problems

The following problems are intended to reinforce the behaviour of conditional statements and loops rather than merely their syntax. Where possible, predict the behaviour of the code before executing it.

24.1 Problem 1 — Truth Values

Without running the code, determine which messages are printed:

```

1 values = []
2
3 if values:
4     print("A")
5 else:
6     print("B")
7
8 if 0:
9     print("C")
10
11 if "0":
12     print("D")

```

Explain each result in terms of Python's truth-value rules.

24.2 Problem 2 — Independent Conditions

Predict the output:

```

1 x = 12
2
3 if x > 0:
4     print("positive")
5
6 if x % 2 == 0:
7     print("even")
8
9 if x > 20:
10    print("large")

```

Then explain how the behaviour would change if the second and third if statements were replaced by elif.

24.3 Problem 3 — Ordering Conditions

Consider:

```

1 score = 0.72
2
3 if score >= 0.5:
4     category = "acceptable"
5 elif score >= 0.7:
6     category = "good"
7 elif score >= 0.9:
8     category = "excellent"
9 else:
10    category = "poor"

```

What value is assigned to category?

Explain why the program does not implement the likely intended classification and rewrite the ordering of the conditions appropriately.

24.4 Problem 4 — Conditional Expression

Given a number n , assign either "even" or "odd" to a variable called parity using a conditional expression.

Do not use an if statement.

24.5 Problem 5 — Point inside a Rectangle

Given a point (x, y) , determine whether it belongs to the closed rectangle

$$[a, b] \times [c, d].$$

Write the condition using chained comparisons.

Then determine how the condition should change if the boundary of the rectangle is excluded.

24.6 Problem 6 — Leap Year

A year is a leap year if it is divisible by 4, except for years divisible by 100, unless they are also divisible by 400.

Write a conditional structure that determines whether a given year is a leap year.

Then attempt to express the same rule using a single Boolean expression.

24.7 Problem 7 — Predicting a for Loop

Without executing the code, determine the final values of total and x:

```
1 total = 0
2
3 for x in [2, 5, 3]:
4     total += x
```

Explain the state of both variables after every iteration.

24.8 Problem 8 — range()

Determine the values generated by each expression without executing them:

```
1 range(5)
2 range(2, 7)
3 range(1, 10, 3)
4 range(10, 1, -2)
5 range(5, 5)
```

For each case, state how many iterations a corresponding for loop would perform.

24.9 Problem 9 — Off-by-One Error

The following code is intended to print the integers from 1 through 10:

```
1 for i in range(1, 10):
2     print(i)
```

Identify the problem and correct it.

Explain the correction in terms of the half-open interval represented by range().

24.10 Problem 10 — Accumulation

Using a for loop, calculate

$$S = \sum_{k=1}^{100} k.$$

Do not use `sum()`.

Then verify the result using the formula

$$\sum_{k=1}^n k = \frac{n(n+1)}{2}.$$

24.11 Problem 11 — Arithmetic Progression

An arithmetic progression is defined by

$$a_k = a_0 + kd.$$

Given `a0`, `d`, and an upper bound `b`, use `range()` and a for loop to calculate the sum of all progression terms not exceeding `b`.

Assume that all values are integers and that `d > 0`.

24.12 Problem 12 — Counting Characters

Given a string, count the number of vowels using an explicit loop.

Do not use `str.count()`.

Your solution should treat uppercase and lowercase letters equivalently.

24.13 Problem 13 — Reverse a String

Given a string `s`, construct its reverse using a for loop.

Do not use slicing with `[::-1]` and do not use `reversed()`.

24.14 Problem 14 — Filtering with continue

Given

```
1 values = [4, -2, 7, -1, 0, 9]
```

use a for loop and continue to print only the non-negative values.

Then rewrite the solution without continue.

Compare the control flow of the two versions.

24.15 Problem 15 — First Matching Value

Given two integers a and b, find the first integer in the closed interval

$$[a, b]$$

that is divisible by 2, 3, and 5.

Terminate the search immediately after finding the first valid number.

If no such number exists, report that no match was found.

24.16 Problem 16 — for-else

Rewrite Problem 15 using a for-else structure so that no additional Boolean variable is required to determine whether a valid number was found.

Explain precisely when the else clause executes.

24.17 Problem 17 — Infinite Loop Diagnosis

Consider:

```
1 x = 1
2
3 while x < 100:
4     x *= 0.5
```

Does this loop terminate?

Explain the evolution of x and identify why the condition never becomes false.

Modify the loop so that it terminates naturally.

24.18 Problem 18 — Iteration until Convergence

Consider the recurrence

$$x_{k+1} = \frac{1}{2} \left(x_k + \frac{a}{x_k} \right),$$

with $a > 0$ and $x_0 > 0$.

This is an iterative method for approximating $\sqrt{a}$.

Starting from

```
1 a = 2
2 x = 1.0
```

repeatedly compute the next approximation until

$$|x_{k+1} - x_k| < 10^{-10}.$$

Use a while loop and count the number of iterations required.

Add a maximum number of iterations as a safety condition.

24.19 Problem 19 — break and State

Predict the output:

```
1 for i in range(10):
2     if i == 4:
3         break
4
5 print(i)
```

Why does i still exist after the loop?

What value would i have if the break statement were removed?

24.20 Problem 20 — continue

Predict the output:

```
1 total = 0
2
3 for i in range(6):
4     if i % 2 == 0:
5         continue
6
7     total += i
8
9 print(total)
```

Explain which values contribute to total.

24.21 Problem 21 — Nested Loops

Without executing the code, determine how many times `print()` is called:

```
1 for i in range(4):
2     for j in range(3):
3         print(i, j)
```

List the first three and final three pairs that are printed.

Generalise the number of inner-loop executions when the outer loop runs m times and the inner loop runs n times.

24.22 Problem 22 — break in Nested Loops

Predict the output:

```
1 for i in range(3):
2     for j in range(4):
3         if j == 2:
4             break
5
6     print(i, j)
```

Explain why `break` does not terminate the outer loop.

24.23 Problem 23 — Matrix Traversal

Given

```
1 matrix = [  
2     [2, 4, 6],  
3     [1, 3, 5],  
4     [7, 8, 9],  
5 ]
```

use nested loops to calculate the sum of all matrix elements.

Then calculate the sum of each row separately.

Do not use `sum()`.

24.24 Problem 24 — Pairwise Comparisons

Given

```
1 values = [3, 8, 2, 7]
```

use nested loops to print every ordered pair (x_i, x_j) .

How many pairs are produced?

Then modify the loops so that each unordered pair of distinct elements is considered exactly once.

For a collection containing n elements, how many such pairs should be produced?

24.25 Problem 25 — `enumerate()`

Given

```
1 errors = [0.31, 0.18, 0.09, 0.04, 0.008]
```

use `enumerate()` to determine the index of the first error smaller than 0.01.

Terminate the loop immediately when the value is found.

24.26 Problem 26 — zip()

Given

```
1 observed = [1, 0, 1, 1, 0, 1]
2 predicted = [1, 1, 1, 0, 0, 1]
```

use `zip()` to count

- correct predictions;
- incorrect predictions.

Then calculate the accuracy

$$\text{accuracy} = \frac{\text{correct predictions}}{\text{total predictions}}.$$

Do not use a machine-learning library.

24.27 Problem 27 — Different Lengths with zip()

Predict what happens:

```
1 x = [1, 2, 3, 4]
2 y = [10, 20]
3
4 for a, b in zip(x, y):
5     print(a, b)
```

How many iterations occur?

Why can this behaviour be dangerous when the two collections are expected to represent corresponding observations?

Investigate how `strict=True` changes the behaviour.

24.28 Problem 28 — Data Validation

Suppose

```
1 values = [0.41, None, 0.72, -0.1, 0.88, None, 0.93]
```

is a collection of measurements.

Process the values according to the following rules:

1. skip None;
2. skip negative values;
3. print every remaining value;
4. stop immediately if a value greater than 0.9 is encountered.

Use continue and break where appropriate.

24.29 Problem 29 — Threshold Classification

Given a sequence of probabilities and a threshold t , convert each probability into a binary prediction according to

$$\hat{y} = \begin{cases} 1, & p \geq t, \\ 0, & p < t. \end{cases}$$

For

```
1 probabilities = [0.12, 0.51, 0.49, 0.87, 0.73]
2 threshold = 0.5
```

construct a list called predictions using an explicit for loop.

Do not use a list comprehension.

24.30 Problem 30 — Early Stopping

Suppose the validation losses from successive training epochs are

```
1 losses = [0.52, 0.44, 0.39, 0.36, 0.355, 0.354, 0.3545, 0.355]
```

Traverse the values in order.

Stop when the current loss is greater than the previous loss and report the epoch at which this first occurs.

Use `enumerate()` and keep track of the previous value explicitly.

Explain why this is a simplified form of an **early-stopping** rule rather than a realistic complete implementation.

24.31 Problem 31 — Search without a Flag

Given

```
1 values = [8, 14, 22, 31, 47]
```

determine whether any value is divisible by both 3 and 5.

Solve the problem using

- a for loop;
- `break`;
- `else`.

Do not introduce a variable such as `found = False`.

24.32 Problem 32 — Reasoning about Control Flow

Without executing the code, determine the exact output:

```
1 for i in range(1, 5):
2     if i == 2:
3         continue
4
5     for j in range(1, 4):
6         if j == 3:
7             break
8
9         print(i, j)
10 else:
11     print("done")
```

Explain separately the effect of

- `continue`;
- the inner `break`;

- the outer else.

In particular, explain why the inner break does or does not prevent the final else from executing.

25 Summary

Python control structures determine the order and repetition of program execution.

Conditional execution is primarily expressed using if, elif, and else. Conditions are evaluated in a Boolean context, so objects may participate in control flow according to their truth values without necessarily being compared explicitly with True or False.

Iteration is primarily expressed through for and while loops. A for loop traverses the values produced by an iterable, whereas a while loop repeats execution as long as a condition remains true.

Statements such as break and continue modify the normal execution of a loop:

- break terminates the current loop;
- continue skips the remainder of the current iteration;
- loop else executes when a loop completes without being terminated by break.

Python provides additional constructs for common iteration patterns. enumerate() associates elements with positions, while zip() supports parallel iteration over corresponding elements from multiple iterables. Unpacking allows structured values to be assigned directly to multiple loop variables.

The appropriate control structure should communicate the logic of the computation rather than merely reproduce its mechanics. Direct iteration is generally preferable to unnecessary indexing, conditions should express their intent clearly, and loops should be used deliberately when no more direct operation already represents the required computation.

In numerical and machine-learning applications, these principles appear repeatedly in tasks such as data validation, sequential processing, convergence checks, searches, threshold-based decisions, and early termination. The syntax itself is simple; the more important skill is being able to reason precisely about the path that execution will follow.